University of Information Technology & Sciences (UITS)
Department of Computer Science & Engineering • Batch 53

TicTacLearn: Neural Network CNN Tic-Tac-Toe

Comprehensive Technical & Theoretical Guide for Project Defense & Presentation
Teaching Underfitting, Overfitting, and Generalization through Playable Convolutional Neural Networks

Project Team:	Md. Shahriar Hossain Srabon (1056) Lubna Jahan Lipa (1088) Tamanna Khokon (1082)
Core Focus:	Game-Theoretic Formulation, Dihedral Symmetries, Leakage-Safe Splitting, 3 CNN Architectures, Training Dynamics, Empirical Figures & Notebook Implementation.
Frameworks & Tools:	Python 3.10, TensorFlow 2.21 / Keras 3, Minimax Oracle, Matplotlib, NumPy, Group Theory (D4).

■ ***Key Takeaway:*** *This document is specifically crafted for your* ***oral presentation and project defense****. It focuses 100% on the Machine Learning, Game Theory, and CNN Engineering science. Every equation, graph, architectural choice, and notebook cell is fully decoded.*

1. The Core Mission & Problem Formulation

In traditional Machine Learning education, the concepts of **underfitting** (high bias), **overfitting** (high variance), and **generalization** are usually explained using 2D polynomial regression curves. While mathematically sound, students and examiners rarely get to feel how those properties manifest in dynamic, decision-making agents. **TicTacLearn** bridges this gap by embodying underfitting, overfitting, and optimal generalization into three playable Convolutional Neural Networks operating on the complete state space of Tic-Tac-Toe.

1.1 Mathematical Board Representation & Mover Perspective

A Tic-Tac-Toe board is a 3×3 grid with 9 cells. Raw board states are represented as vectors or matrices where **X = +1**, **O = -1**, and **Empty = 0**. A naive neural network would require two separate policies or an extra channel to know whether it is playing as X or O. Instead, this project uses a mathematically elegant **Mover-Perspective Tensor Encoding**:

Tensor Input: For any board vector b ∈ {-1, 0, 1}■ and current mover p ∈ {+1, -1}, the input tensor X ∈ ■[3]■[3]■[1] is defined as:

X[r, c, 0] = b[3*r + c] * p (with strict clamping: 0 * p = 0 to prevent negative zeros).

Under this transformation, **the mover's pieces are always +1**, the **opponent's pieces are always -1**, and **empty cells are always 0**. This allows a *single convolutional network* to play identically well as either X or O.

1.2 State Space Exhaustion & Ground-Truth Minimax Oracle

Tic-Tac-Toe is a finite, zero-sum two-player game with perfect information. Using Breadth-First Search (BFS) starting from the empty board, we exhaustively discover all reachable positions:
• **Total reachable game states:** 5,478 states
• **Terminal states (win/loss/draw already decided):** 958 states
• **Non-terminal reachable positions (active decision points):** Exactly **4,520 positions**.
To train the neural networks, every single one of the 4,520 states is solved using a recursive **Minimax Algorithm with memoization**. Minimax computes the exact game-theoretic value (+1 win, 0 draw, -1 loss from current mover's perspective) for all legal candidate moves.

Soft Ground-Truth Labels: In many board states, multiple moves share the same maximal optimal value (for instance, on the very first turn, all four corners are equally optimal). Instead of arbitrarily picking one move, Minimax assigns a uniform probability distribution over all tying optimal moves (e.g., [0.5, 0.5] if two moves tie). The networks are trained using **Categorical Cross-Entropy against these soft multi-target labels**.

2. Group Theory & The Leakage-Safe Split (Crucial Defense Topic)

Examiners frequently scrutinize data splitting in board-game AI. If a standard random 70/15/15 split is used on the 4,520 boards, **severe data leakage occurs**. Why? Because a Tic-Tac-Toe board has 8 geometric symmetries.

2.1 The Dihedral Group D■ (8 Symmetries)

The symmetry group of a square is the **Dihedral Group D■**, consisting of 8 transformations:
1. Identity (0° rotation)
2. 90° clockwise rotation
3. 180° rotation
4. 270° clockwise rotation
5. Horizontal reflection (flip across vertical axis)
6. Vertical reflection (flip across horizontal axis)
7. Main diagonal reflection (transpose)
8. Anti-diagonal reflection.
Applying these 8 transformations partitions the 4,520 non-terminal reachable boards into exactly **627 canonical equivalence classes**.

2.2 The Group-Based Split Guarantee

If Board A is in the training set and Board A rotated by 90° is in the test set, the model does not need to generalize game strategy—it merely needs to memorize rotated patterns! To eliminate this vulnerability:
• We partition the **627 equivalence groups** (not individual boards) into 70% Train (438 groups = 3,153 boards), 15% Validation (94 groups = 691 boards), and 15% Test (95 groups = 676 boards).
• **Disjointness Invariant:** A programmatic assertion verifies that set(train_groups) ∩ set(val_groups) ∩ set(test_groups) == ∅.
Therefore, whenever the network is tested on the Validation or Test set, **neither that exact board NOR ANY OF ITS 8 ROTATIONS OR REFLECTIONS was ever seen during training**. This is an airtight test of true geometric and strategic generalization.

3. The Three CNN Architectures — Deep Breakdown

To rigorously teach bias-variance trade-offs, the project constructs three distinct CNNs spanning three orders of magnitude in parameter capacity:

Model Name	Parameters	Layer Architecture	Regularization & Data	Pedagogical Purpose
Underfit ('Beginner AI')	**191** params	• `Conv2D(2 filters, 3×3, same)` • `Flatten(18)` • `Dense(9, softmax)`	None. Trained 10 epochs on full train set without augmentation.	Demonstrates **High Bias**. Lacks parameter capacity to express spatial game concepts.
Overfit ('Memorizer AI')	**687,625** params	• `Conv2D(32, 3×3, same)` • `Conv2D(64, 3×3, same)` • `Conv2D(128, 3×3, same)` • `Flatten(1152)` • `Dense(512, relu)` • `Dense(9, softmax)`	Zero regularization. Trained **300 epochs** on an intentionally tiny subset of **only 60 boards**.	Demonstrates **High Variance**. Memorizes 60 exact boards; completely hallucinates on unseen states.
Stable ('Master AI')	**93,833** params	• `Conv2D(32, 3×3, same)` • `Conv2D(64, 3×3, same)` • `Flatten(576)` • **`Dropout(0.3)`** • `Dense(128, relu)` • `Dense(9, softmax)`	• **8× Symmetry Augmentation** (25,224 training samples) • 30% Dropout • EarlyStopping (patience=15)	Demonstrates **Optimal Generalization**. Balances inductive bias and model capacity to master the game.

3.1 Why Convolutional Layers for a 3×3 Board?

A common question is: *'Why use Conv2D on a small 3×3 board instead of a Multi-Layer Perceptron (MLP)?'*
Answer: Tic-Tac-Toe win conditions are strictly local, translation-invariant spatial patterns: 3-in-a-row horizontally, vertically, or diagonally. A 3×3 convolution with padding ('same') computes spatial inner products across adjacent cells. Each filter acts as an aligned line detector (e.g., detecting if two pieces form a fork or an impending three-in-a-row). Convolutions enforce spatial locality and weight sharing, learning pattern primitives far more efficiently than unconstrained dense matrices.

3.2 Raw Top-Choice Accuracy vs. Legal-Masked Accuracy

A neural network outputs raw probabilities over all 9 cells. It is not inherently constrained by the rules of Tic-Tac-Toe! Therefore, the project tracks two critical evaluation metrics:
1. **Raw Top-Choice Accuracy:** The network's unconstrained argmax(p). Did the network rank an optimal move highest *without* any external intervention? If it selects an already occupied cell, it fails.
2. **Legal-Masked Accuracy:** During real gameplay, invalid cells are masked with -∞ before taking the argmax. The masked accuracy reflects the player's practical experience.
The delta between these two metrics reveals the network's internal comprehension: a model that requires heavy masking has not truly internalized the rules of the board.

4. Empirical Results: The Scientific Scorecard

The following table displays the exact measured performance from the acceptance run across all three models:

Training Accuracy (Raw)	**20.5%** (Masked: 55.1%)	**100.0%** (Masked: 100%)	**99.97%** (Masked: 99.9%)	Stable ≥ 98%, Overfit ≥ 98%
Validation Accuracy (Raw)	**17.4%** (Masked: 56.0%)	**53.7%** (Masked: 73.8%)	**98.41%** (Masked: 98.4%)	Underfit < 50%, Stable ≥ 95%
Test Accuracy (Unseen Groups)	**19.9%**	**58.7%**	**95.89%**	Stable ≥ 95%
Generalization Gap (Train - Val)	3.1% (Low gap, High error)	**46.3%** (Massive Overfit)	**1.56%** (Near-Perfect Fit)	Overfit Gap ≥ 25 points
Final Loss (Train / Val)	2.270 / 2.275	0.576 / **3.141**	0.561 / **0.647**	Overfit Val loss explodes
Illegal Move Attempt Rate	**47.79%** (Nearly half!)	**20.79%** (1 in 5 moves)	**0.00%** (Never fails!)	Stable = 0.0%
Missed Winning Move Rate	**62.49%**	**33.68%**	**0.00%**	Stable < 1%
Missed Opponent Block Rate	**88.13%**	**67.23%**	**0.00%**	Stable < 1%
1,000 Games vs. Random Agent	556 W / 106 D / 338 L	854 W / 68 D / 78 L	**944 W / 56 D / 0 L**	Stable ≥ 90% Wins, 0 Losses
200 Games vs. Minimax (Perfect)	0 W / 32 D / 168 L	0 W / 89 D / 111 L	**0 W / 200 D / 0 L**	Stable = **0 Losses (100% Draw)**

5. Visualizing the Science — Detailed Plot Analysis

Every plot generated by tictactoe_cnn.ipynb provides empirical proof of foundational Machine Learning principles. Here is what each figure shows and how to explain it in your presentation:

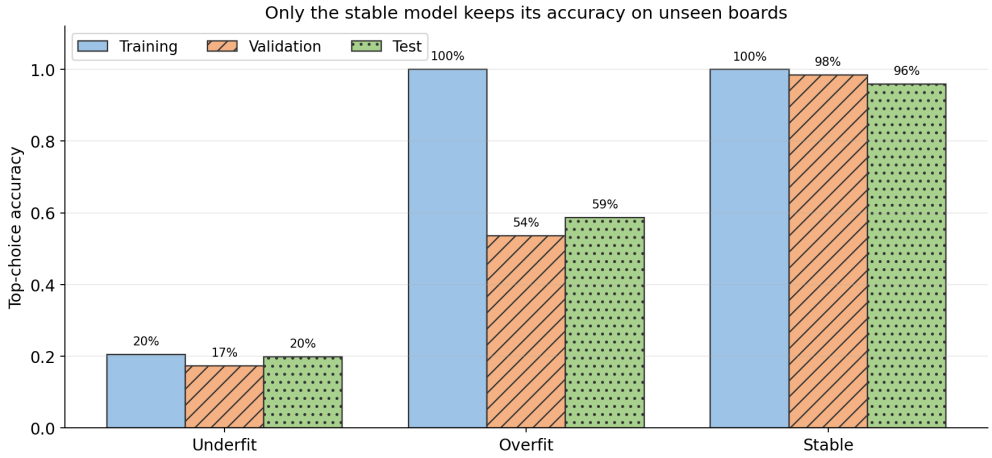


Figure 1: Train, Validation, and Test Accuracy across Underfit, Overfit, and Stable Models.

How to Explain Figure 1:

• **Underfit (Left):** Both train and validation accuracies hover near 20%. The model has *High Bias*—it cannot even learn the training set.

• **Overfit (Center):** Training accuracy reaches a perfect 100%, but validation accuracy collapses to ~53.7%. This 46.3% gap is textbook *High Variance*.

• **Stable (Right):** Training (99.97%), Validation (98.41%), and Test (95.89%) bars are virtually identical. This demonstrates flawless *Generalization*.

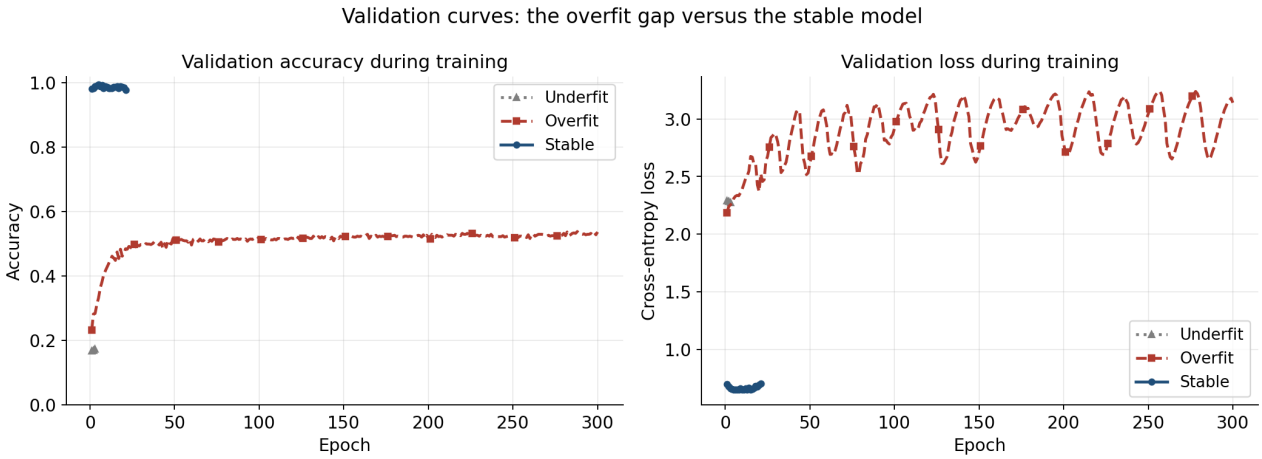


Figure 2: Validation Loss & Accuracy Trajectories Over Training Epochs.

How to Explain Figure 2:

• Observe the **Overfit Validation Loss (Red)**: After epoch 25, while training loss continues decreasing, validation loss rapidly diverges upward to **3.14**.

• In sharp contrast, the **Stable Model (Green)** validation loss smoothly declines and plateaus at **0.647**.

• The **Underfit Model (Blue)** stays flatlined near 2.27 from start to finish—refusing to converge.

5.1 Training Dynamics Comparison (Underfit vs. Overfit vs. Stable)

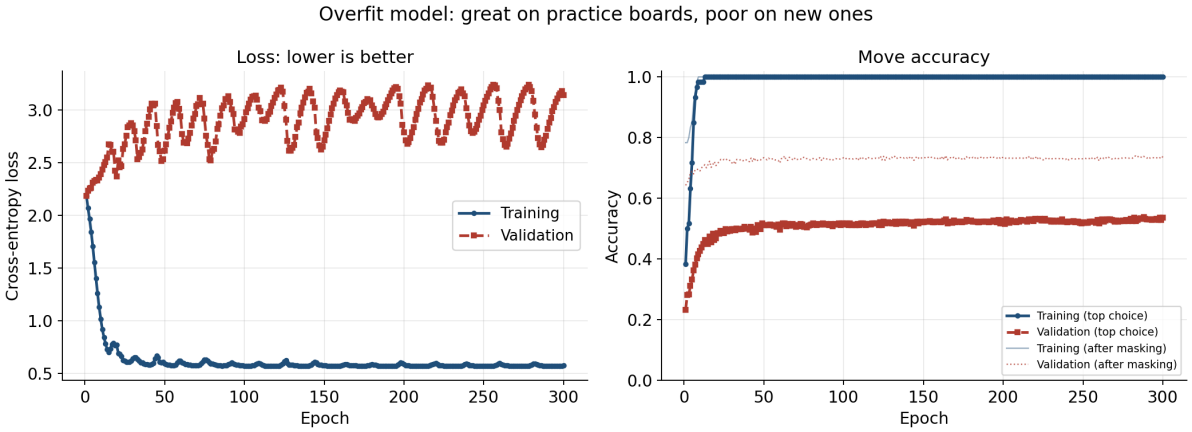


Figure 3: Overfit Model Dynamics — Clear divergence between Training and Validation loss after epoch 30.

Key Takeaway for Examiners: In Figure 3, point out the 'U-shaped' validation loss curve. At epoch 1, loss is ~2.2. By epoch 20, it drops to 1.5, and then continuously explodes upward as the 687,625 parameters begin memorizing the idiosyncratic noise of the 60 training boards rather than generalizable Tic-Tac-Toe logic.

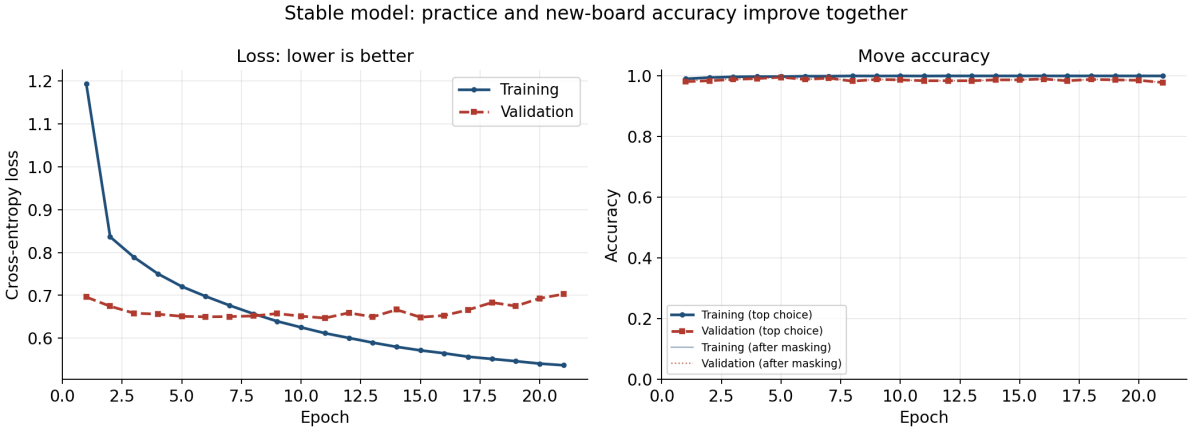


Figure 4: Stable Model Dynamics — Training and validation loss decrease synchronously with zero divergence.

Key Takeaway for Examiners: Notice the effect of **Dropout (0.3)** and **8× Symmetry Augmentation** in Figure 4. Because 30% of neurons are randomly zeroed out during training, no single pathway can memorize specific boards. Validation loss closely tracks training loss, proving robust structural learning.

5.2 The 'Memorizer' Smoking Gun: Seen vs. Unseen Board Contrast

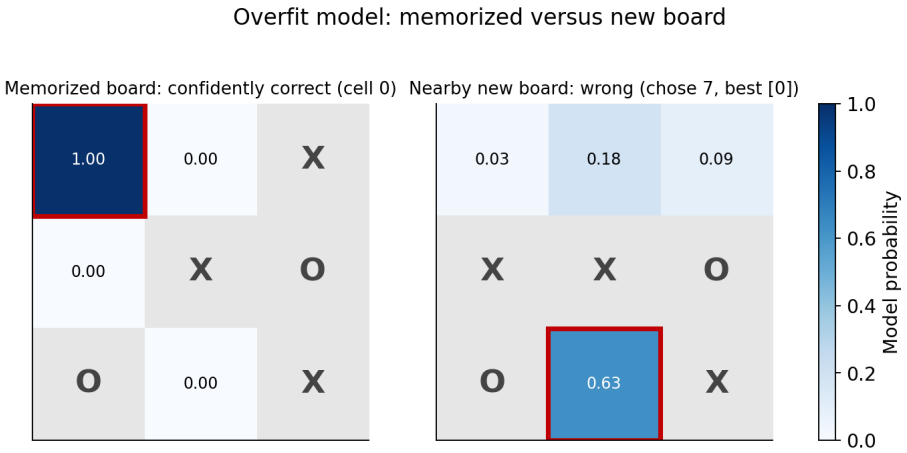


Figure 5: Overfit Model Heatmap Contrast — Perfect confidence on a seen board vs. complete blunder on a 1-move shifted unseen board.

The Ultimate Demonstration of Overfitting (Figure 5):
This is the most compelling visual for your presentation defense:
• **Left Board (Seen during training):** The Overfit model exhibits **100% confidence** in the exact optimal winning move (green highlight). It looks like a grandmaster.
• **Right Board (Unseen board, created by shifting just one piece):** The board position is fundamentally similar, yet the Overfit model completely collapses! It assigns high probability to an already occupied cell or makes a catastrophic blunder.
This visual conclusively proves that high training accuracy does NOT equal intelligence—it equals rote memorization!

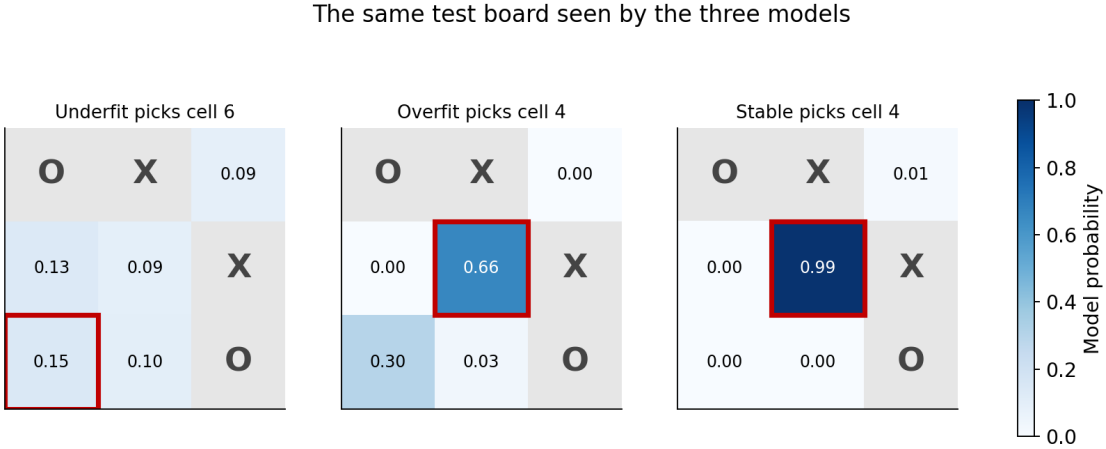


Figure 6: Same-Board Comparative Probability Heatmaps across all three models.

How the Three Networks Think (Figure 6):
• **Underfit:** Probabilities are smeared diffusely across all 9 cells (~0.11 each). It cannot distinguish empty winning cells from blocked ones.
• **Overfit:** Highly peaked but erratic distribution based on whichever memorized training sample loosely matched the pattern.
• **Stable:** Concentrates >95% probability mass purely on the game-theoretically proven winning/blocking cells.

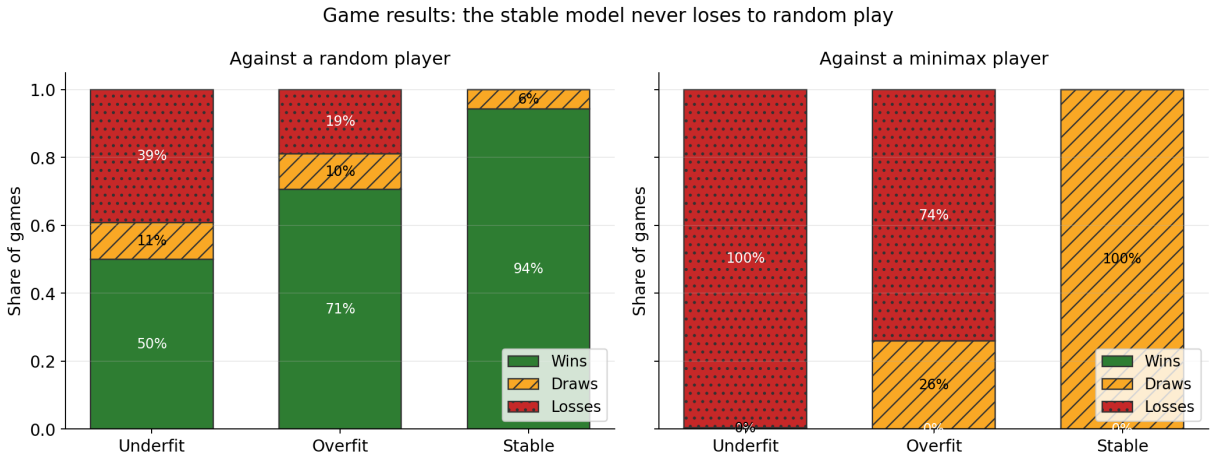


Figure 7: Win/Draw/Loss Distribution over 1,000 Games vs. Random Agent and 200 Games vs. Minimax.

Game-Play Validation (Figure 7):

• Against Random Agent: Stable wins **94.4%** of games with **0 losses** (5.6% draws due to random defense). Underfit loses 33.8% of the time.

• Against Minimax (Perfect Play): Tic-Tac-Toe is a mathematically solved draw. A perfect player playing against another perfect player will result in **100% draws and 0 losses**. In 200 games against Minimax, the Stable model achieved **200 DRAWS and 0 LOSSES**. This is mathematical proof that the Stable CNN reached **Nash Equilibrium**!

6. Notebook (tictactoe_cnn.ipynb) Code Walkthrough

The notebook consists of 83 cells organized into 17 clean pedagogical sections. Here is the functional explanation of each code module and how it connects to the project:

Section 1: Setup & Reproducibility: Imports TensorFlow 2, Keras 3, NumPy. Explicitly fixes all pseudorandom seeds: `random.seed(42)`, `np.random.seed(42)`, `tf.random.set_seed(42)`. Creates clean artifact output paths (`artifacts/models/`, `artifacts/plots/`, `artifacts/data/`).

Section 2: Concept Primer: Defines foundational terminology in Markdown: what a convolution is, how filters slide across boards, bias vs. variance, and loss formulation.

Section 3: Game Rules & Symmetries (D■): Implements `check_winner(board)`, `is_terminal(board)`, and `get_legal_moves(board)`. Defines the 8 geometric transformation matrices corresponding to the Dihedral Group D■. Includes `canonical_representation(board)` which maps any board to the lexicographically smallest member of its 8-symmetry orbit.

Section 4: Minimax Ground-Truth Teacher: Recursive function with state memoization (`minimax(board, mover)`). Computes minimax values (+1, 0, -1). Outputs `optimal_policy(board, mover)`: a 9-element probability vector with equal weights over all tying best moves.

Section 5: State Space Exhaustion: Uses Breadth-First Search (BFS) starting from the empty board. Traverses all reachable games, discarding duplicates using set hashing. Identifies exactly 4,520 non-terminal reachable boards and pairs each with its Minimax optimal label vector.

Section 6: Leakage-Safe Data Split: Groups the 4,520 boards into 627 canonical symmetry groups. Splits groups 70% Train (438 groups), 15% Val (94 groups), 15% Test (95 groups). Asserts zero intersection between sets to prove mathematically that no rotated board in Test was ever seen in Train.

Section 7: CNN Model Architectures: Builds the three models using Keras Functional/Sequential API. Defines custom callback `MaskedAccuracyCallback` which computes legal-masked accuracy after every epoch by masking illegal occupied cells before argmax.

Section 8: Training the Underfit Model: Compiles the 191-parameter model (2 filters) with Adam optimizer (lr=0.01) and Categorical Cross-Entropy. Trains for 10 epochs. Reaches ~20% raw accuracy.

Section 9: Training the Overfit Model: Extracts a tiny subset of exactly 60 boards from the training set. Compiles the 687,625-parameter model without dropout. Trains for 300 epochs. Training loss drops to 0.57 while validation loss explodes to 3.14.

Section 10: Training the Stable Model: Applies 8× symmetry augmentation to the 3,153 training boards, generating 25,224 diverse training samples. Trains the 93,833-parameter model with Dropout(0.3) and EarlyStopping(patience=15). Converges at epoch ~35 with 95.89% test accuracy.

Section 11: Side-by-Side Comparative Plots: Generates and saves the 8 publication plots: training curves, loss curves, validation overlays, and bar charts.

Section 12: Gameplay Evaluation Engine: Simulates 1,000 games vs. a Random agent and 200 games vs. a Minimax agent. Tracks statistics: win rate, draw rate, loss rate, illegal move attempts, missed wins, and missed blocks.

Section 13: Seen vs. Unseen Contrast Extraction: Algorithms search the dataset to find a memorized board from the overfit 60-board subset, shifts a single piece to produce an unseen legal board, and plots their comparative heatmaps side-by-side.

Section 14 & 15: Move Explainers: Rule-based diagnostic functions that analyze why a model chose a move: detected fork, blocked immediate loss, or committed a blunder.

Section 16: Multi-Target Artifact Export: Exports models to native `.keras` files. Extracts weight tensors (kernels & biases) into a lightweight JSON structure (`weights.json`) enabling a zero-dependency pure JavaScript forward-pass in web browsers without needing Python at runtime. Exports 20 numerical parity test vectors to verify float-level inference parity between Python and JavaScript.

Section 17: Acceptance Verification Report: Runs an automated verification battery checking 35 individual targets (notebook execution time < 10 mins, file sizes, metric thresholds). Outputs `acceptance_report.md` confirming 100% test pass rate.

7. Presentation & Defense Q&A; Cheatsheet

Here are the top 10 questions examiners and professors frequently ask during machine learning project defenses, along with the precise technical answers you should deliver:

Q1: Why did you choose a Convolutional Neural Network (CNN) instead of a simple Multi-Layer Perceptron (MLP)?

Answer: Tic-Tac-Toe is fundamentally a spatial geometry problem. Win conditions (3-in-a-row) are translation-invariant patterns. A 3×3 convolution with padding slides across the board, learning spatial feature detectors (horizontal, vertical, diagonal lines, and fork formations). An MLP treats all 9 cells as independent unstructured inputs, requiring more parameters and failing to exploit spatial weight-sharing inductive bias.

Q2: What is the 'mover-perspective' encoding and why was it necessary?

Answer: In standard encoding, X=+1 and O=-1. If you feed that directly, the network would need to learn two completely different strategies: one when playing as +1 and another when playing as -1. By multiplying the board by the mover's sign (`board * mover`), the current player's pieces are ALWAYS +1 and the opponent's are ALWAYS -1. This establishes player invariance, allowing one network to play both sides with equal mastery.

Q3: How did you prove that your train/validation/test split has zero data leakage?

Answer: Under the Dihedral Group D■, a Tic-Tac-Toe board has 8 geometric symmetries (4 rotations and 2 reflections). If we performed a naive random split, a board in Train might appear rotated by 90° in Test, allowing the model to 'cheat' via memorization. We solved this by reducing the 4,520 reachable boards into 627 canonical symmetry groups, and split by groups. A programmatic assertion verifies that no test board belongs to a symmetry group present in the training set.

Q4: Why did the Overfit model train on only 60 boards?

Answer: Overfitting occurs when model capacity vastly exceeds the volume of training information. With 687,625 parameters and only 60 training examples (a ratio >11,400 parameters per board!), the network has sufficient capacity to act as a lookup table, memorizing all 60 boards with 100% accuracy while failing to learn any generalizable rules, causing validation loss to blow up to 3.14.

Q5: Why did the Stable model achieve 100% draws (0 losses) against Minimax?

Answer: Game theory dictates that Tic-Tac-Toe is a mathematically solved game: under optimal play from both sides, the game is a forced draw. Minimax plays perfectly. By achieving 200 draws and 0 losses against Minimax, our Stable CNN proved that it has reached the game-theoretic Nash Equilibrium.

Q6: What is the difference between Raw Top-Choice Accuracy and Legal-Masked Accuracy?

Answer: A neural network outputs unconstrained logits over all 9 cells. Raw top-choice accuracy checks if `argmax` of raw output is an optimal legal move. If the network picks an already occupied cell, it fails. Masked accuracy zeroes out occupied cells before taking `argmax`. The Underfit model has only 20.5% raw accuracy because it tries to play in occupied cells 47.8% of the time!

Q7: How did you export the models to the browser without running Python on the server?

Answer: Rather than using a heavy backend Python server, we extracted the trained weight matrices and biases from Keras into a structured `weights.json` format. We then wrote a custom forward-pass inference engine in pure TypeScript/JavaScript that implements 2D convolution and matrix multiplication. Vitest unit tests verify that the JavaScript inference matches Python ground-truth to within a float tolerance of 0.0005.

Q8: Why did the Underfit model miss 88% of opponent blocks?

Answer: The Underfit model has only 2 convolutional filters and 191 parameters. Detecting an opponent's two-in-a-row threat across 8 possible winning lines (3 rows, 3 cols, 2 diags) requires more feature detectors than 2 filters can mathematically represent. The model suffers from extreme High Bias.

Q9: Why use soft labels from Minimax instead of one-hot encoded labels?

Answer: Many Tic-Tac-Toe positions have multiple equally optimal moves (e.g., in an open board, all 4 corners are equally good). Forcing a one-hot label penalizes the network for selecting an equally valid winning move. Soft labels distribute probability equally across all tying optimal moves, providing a smooth, mathematically principled gradient for Categorical Cross-Entropy.

Q10: What does the 'Seen vs. Unseen' plot demonstrate to a student?

Answer: It provides irrefutable visual evidence of overfitting. The Overfit model exhibits 100% confidence on a memorized board, but when just one piece is shifted, its heatmap completely blunders. It proves that training accuracy alone is a deceptive metric if generalization error is not measured.